

Krótkie podsumowanie sprintu (Inżynieria Oprogramowania)

Karyna Rudenko, Viktor Zasimovych

23 czerwca 2026

1 Co zostało zrobione

Konfiguracja budowania (CMake)

- Rozbudowanie pliku `CMakeLists.txt` o automatyczne wykrywanie plików źródłowych dla katalogów `src` oraz `util`.
- Dodanie mechanizmu automatycznego generowania pliku rejestru komponentów na etapie konfiguracji CMake.
- Skonfigurowanie odrębnego celu budowania `UTIL` dla narzędzi pomocniczych (folder `util`), niezależnego od głównego pliku wykonywalnego `program`.
- Podłączenie bibliotek zewnętrznych: `SDL2`, `SDL2main`, `SDL2_image`, `SDL2_ttf`, `SDL2_mixer` oraz `yaml-cpp`.

Narzędzie generatora komponentów (`util/CompGenerator.cpp`)

- Implementacja samodzielnego programu konsolowego `CompGenerator`, który na podstawie nazwy istniejącej klasy bazowej (np. `Camera`, `Light`) oraz nazwy nowego komponentu generuje automatycznie:
 - plik nagłówkowy nowej klasy dziedziczącej po klasie bazowej, z zachowaniem sekcji `public/private` oraz nadpisaniem metod oznaczonych jako `virtual` (z dodanym specyfikatorem `override`),
 - odpowiadający plik `.cpp` zawierający szkielet implementacji oraz funkcję rejestrującą komponent w `ComponentFactory`.
- Mechanizm parsowania pliku nagłówkowego klasy bazowej i podstawowa transformacja tekstu w celu wygenerowania poprawnej składni C++.

Testy jednostkowe

- Opracowano zestaw 61 automatycznych testów jednostkowych
- do testów został użyty framework – **Google Test**

```
# Kompilacja (Linux/g++, wymagana biblioteka libgtest-dev)
g++ -std=c++17 -I include -I yaml-cpp-0.8.0/include \
    -I /usr/src/gtest/include \
    tests/test_engine.cpp \
    src/BasicGeometry.cpp src/ComponentFactory.cpp src/Component.cpp \
    /usr/src/gtest/build/lib/libgtest.a \
    /usr/src/gtest/build/lib/libgtest_main.a \
    -lpthread -o tests/run_tests

# Uruchomienie
./tests/run_tests
```

Vector2D – 11 testów

- Konstruktor_ZapisujePola – poprawne przypisanie pól `m_u`, `m_v`.
- Dodawanie – sumowanie składowych dwóch wektorów 2D.
- OdejmowanieWektora – odejmowanie składowych dwóch wektorów 2D.
- OdejmowanieSkalarU – odejmowanie skalaru od obu składowych.
- MnozeniePrzezSkalar – mnożenie składowych przez skalar.
- DzieleniePrzezSkalar – dzielenie składowych przez skalar.
- Dlugosc_TrojkaPytag – długość wektora (3,4) równa 5 (trójka pitagorejska).
- Dlugosc_ZerowegoWektora – długość wektora zerowego wynosi 0.
- Normalize_DajeJednostkowy – po normalizacji długość wynosi 1.
- Normalize_NieZmieniaKierunkuDodatniego – kierunek wektora pionowego (0,5) po normalizacji to (0,1).
- Normalize_WektorZerowy_BezKraksy – normalizacja wektora (0,0) nie rzuca wyjątku.

Vector3D – 12 testów

- Konstruktor_ZapisujePola – poprawne przypisanie `m_x`, `m_y`, `m_z`.
- DomyslnyKonstruktor_DajeZera – domyślny konstruktor inicjuje wektor zerowy.
- Dodawanie, OdejmowanieWektora – operacje składowe.
- MnozeniePrzezSkalar, DzieleniePrzezSkalar – skalowanie wektora.
- Dlugosc_TrojkaPytag3D – długość wektora (1,2,2) równa 3.
- Dlugosc_WektoraJednostkowego – długość (1,0,0) wynosi 1.
- Normalize_DajeJednostkowy – po normalizacji długość wynosi 1.

- `Normalize_KierunekX` – wektor $(5,0,0)$ po normalizacji to $(1,0,0)$.
- `Rotate_Zerowy_BezZmiany` – obrót o $(0^\circ, 0^\circ, 0^\circ)$ nie zmienia wektora.
- `Rotate_360StopniDajeOryginal` – obrót o 360° wokół osi Y zwraca oryginalny wektor (tolerancja $1e-3$).

Funkcje geometryczne (`dotProduct`, `crossProduct`) – 8 testów

- `DotProduct_ProstopadleWektory` – iloczyn skalarny wektorów prostopadłych wynosi 0.
- `DotProduct_RownolegleWektory` – iloczyn skalarny równoległych wektorów $(2,0,0) \cdot (3,0,0) = 6$.
- `DotProduct_PrzeciwnieWektory` – iloczyn skalarny przeciwnych wektorów wynosi -1.
- `DotProduct_OgolnyPrzyklad` – $(1,2,3) \cdot (4,5,6) = 32$.
- `CrossProduct_DwaWektory_Prostopadlosc` – $x \times y = (0,0,1)$.
- `CrossProduct_TrzeciBazowy` – $y \times x = (0,0,-1)$.
- `CrossProduct_TrzyPunkty_NormalTriangla` – iloczyn wektorowy trójkąta w płaszczyźnie XY daje wektor normalny równoległy do osi Z.
- `CrossProduct_RownolegleDajeZerowy` – iloczyn wektorowy równoległych wektorów wynosi $(0,0,0)$.

Matrix4x4 – 10 testów

- `GetIdentMatrix_MaJedynkiNaPrzekatnej` – macierz identyczności ma 1 na przekątnej, 0 poza nią.
- `MatrixXMatrix_IdentPrzezIdent_DaIdent` – $I \cdot I = I$.
- `MatrixXMatrix_IdentPrzezDowolna_DajeTaSama` – $I \cdot M = M$ dla dowolnej macierzy M.
- `GetTransMatrix_UstawiaTranslacje` – macierz translacji o $(3,5,7)$ ma te wartości w ostatnim wierszu.
- `VectorXMatrix_IdentNieZmieniaWektora` – mnożenie wektora przez I nie zmienia go.
- `QuickInverse_IdentPrzezIdentDaIdent` – odwrotność macierzy I to I.
- `QuickInverse_OdwracaMacierzObrotu` – $R \cdot R^{-1} \approx I$ (tolerancja $1e-3$).
- `GetRotMatrix_Zero_DaIdentycznosc` – macierz obrotu o $(0^\circ, 0^\circ, 0^\circ)$ jest macierzą identyczności.
- `GetRotMatrix_90Y_ObraCaOsX` – obrót wektora $(1,0,0)$ o 90° wokół Y zmienia składowe x lub z.

- `OperatorMnozenia_Przeziennosc_IdentPrzezIdent` – operator $*$ macierzy: $I*I$ daje 1 na przekątnej.

Triangle3D – 3 testy

- `Konstruktor_ZapisujePunkty` – wierzchołki trójkąta są poprawnie przechowywane.
- `Normalna_ProstopadlaDoPlaszczyzny` – normalna trójkąta w XY jest prostopadła do jego krawędzi i ma długość 1.
- `KonstruktorZNormalna_UzywaJeyPoBezposredniej` – konstruktor z podaną normalną normalizuje ją poprawnie.

Triangle2D – 4 testy

- `Konstruktor_ZapisujePunkty` – wierzchołki 2D są poprawnie przechowywane.
- `Normalna_JestZnormalizowana` – automatycznie obliczona normalna ma długość 1.
- `ComputeNormal_ProstopadlaDoKrawedzi` – normalna jest prostopadła do krawędzi AB trójkąta.
- `KonstruktorZNormalna_Normalizuje` – konstruktor normalizuje podaną normalną $(0,10) \rightarrow (0,1)$.

Przecięcie z płaszczyzną (`vectorXplainIntersec`) – 3 testy

- `SrodekOdcinka_NaPlaszczyznie` – punkt przecięcia ze środkiem odcinka leży na płaszczyźnie $y=0$, parametr $t=0.5$.
- `PrzeciecieWEkscentrycznym_PunkcieOdcinka` – dla płaszczyzny $x=3$ i odcinka $[0,6]$ punkt przecięcia leży w $x=3$.
- `Parametr_t_Rowny1_GdyKoniecNaPlanie` – gdy koniec odcinka leży na płaszczyźnie, parametr $t=1$.

getPointAtMatrix – 2 testy

- `MacierzNieJestZerowa` – wygenerowana macierz widoku kamery jest niezerowa.
- `PozycjaPrzechowywanaWOstatnimWierszu` – pozycja kamery $(3,5,7)$ jest zapisywana w ostatnim wierszu macierzy.

ComponentFactory (ECS) – 8 testów

- `Rejestracja_ZwracaTrueGdyNowaKlasa` – rejestracja nowego komponentu zwraca `true`.
- `Rejestracja_ZwracaFalseGdyDuplikat` – ponowna rejestracja tej samej nazwy zwraca `false`.
- `Create_ZnanaNazwa_ZwracaNieNullptr` – tworzenie zarejestrowanego komponentu zwraca niepusty wskaźnik.
- `Create_NieznanaKlasa_ZwracaNullptr` – tworzenie nieznanego komponentu zwraca `nullptr`.
- `Create_ZwracaPoprawnyTypKomponentu` – zwrócony wskaźnik można bezpiecznie rzutować na właściwy typ (`dynamic_cast`).
- `Create_DwaRozneKomponenty` – fabryka poprawnie rozróżnia i tworzy różne typy komponentów.
- `Create_KazdyCallZwracaNowaInstancje` – każde wywołanie `create()` tworzy odrębną instancję (różne adresy).
- `Rejestracja_WieluKomponentow_WszystkieDostepne` – po rejestracji 5 różnych komponentów każdy z nich można utworzyć.

1.0.1 Dalszy plan

Sprint zamykał zaplanowany zakres prac projektu zgodny z kartą projektu. Dalszy rozwój silnika mógłby obejmować m.in. testy integracyjne renderera, wsparcie dla tekstur w renderze oraz edytor sceny.

[Repozytorium GitHub](#)

[Trello](#)